

GDGoC

Week5-task1

2025.01.06

기존 모델 생성 후 재사용 하여 Transfer Learning을 수행하고,
레이어를 동결해 모델의 학습 속도를 높일 수 있다.

컴퓨터공학과

20214111 김동환

목차

1	문제의 요구사항 기술	03
	- 문제를 완벽하고 모호함 없이 기술 - 문제가 무엇을 요구하는지 기술	
2	문제 분석	04
	- 문제의 입력과 출력을 식별, 데이터 구조 정의 - 입출력 변수 정의 및 관계식 도출	
3	알고리즘 분석	05
	- 프로그램 구조도(함수 /모듈간 관계도) - 모듈별 알고리즘 분석	
4	프로그램 작성(코딩)	06
	코딩 및 주석 작성	
5	프로그램 테스트 및 검증	09
	입력 및 출력 데이터에 기준하여 프로그램 검증	

문제 요구사항 기술

기존 모델을 생성 한 후 활용하여 Transfer Learning 수행

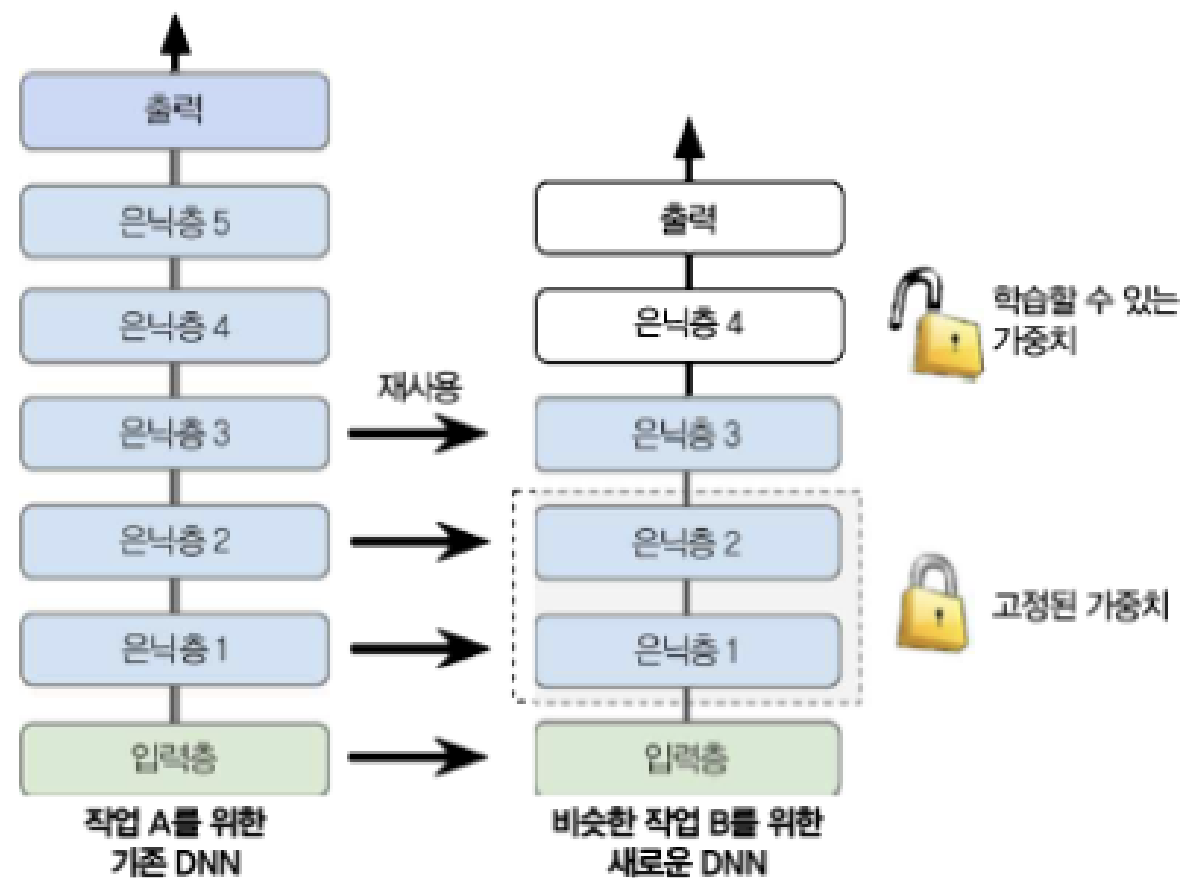


그림 11-4 사전훈련된 층 재사용하기

NOTE 만약 원래 문제에서 사용한 것과 크기가 다른 이미지를 입력으로 사용한다면 원본 모델에 맞는 크기로 변경하는 전처리 단계를 추가해야 합니다. 일반적으로 전이 학습은 저수준 특성이 비슷한 입력에서 잘 작동합니다.

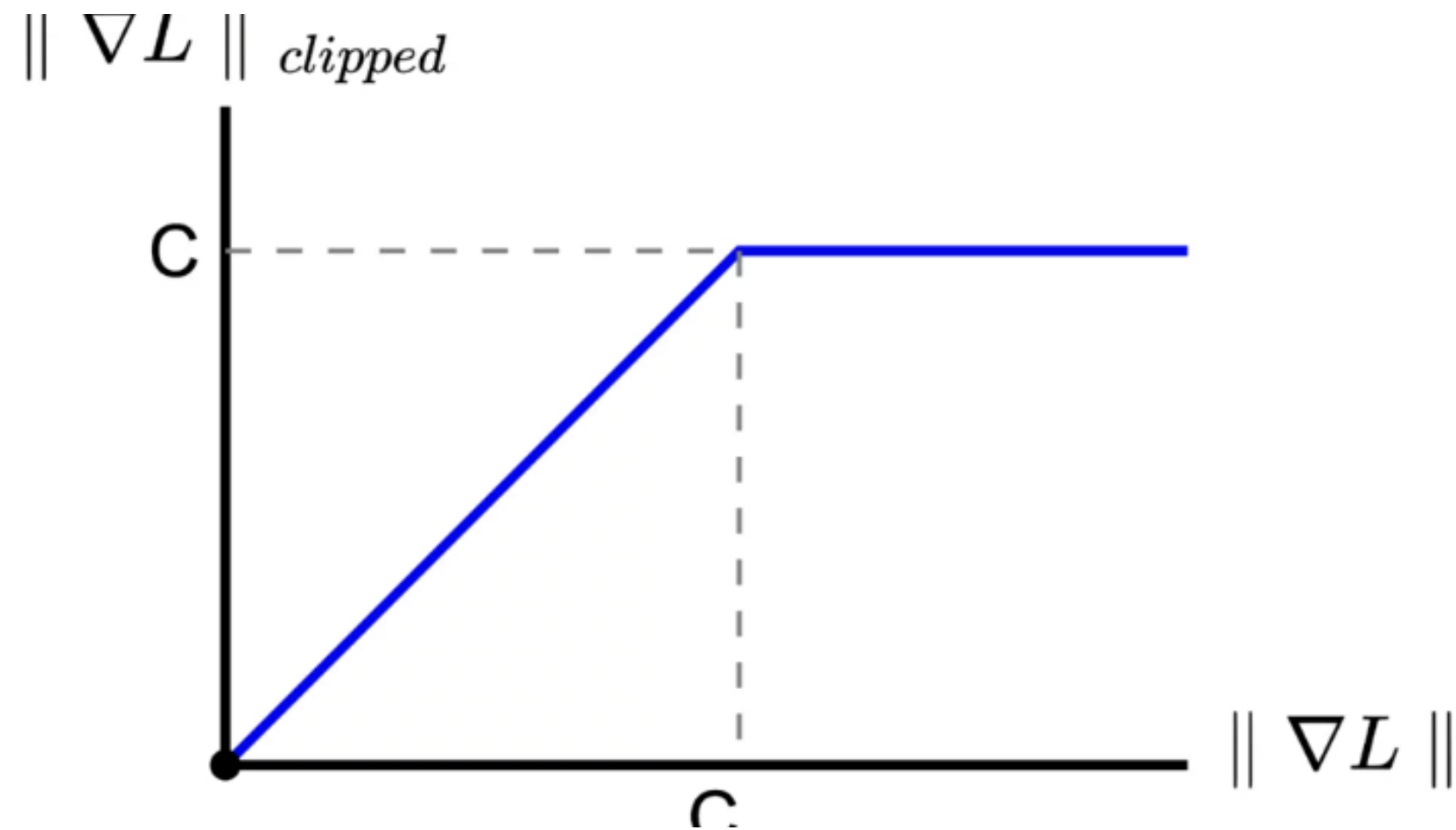
문제 정의

- 기존 모델을 생성하기 위해 코드 실행
- 기존 모델을 활용해 입력층과 은닉층, 출력층을 추가 및 재정의
- 은닉층 레이어를 동결시켜 학습 시간을 줄임
- 확장된 모델을 가지고 정확도 측정

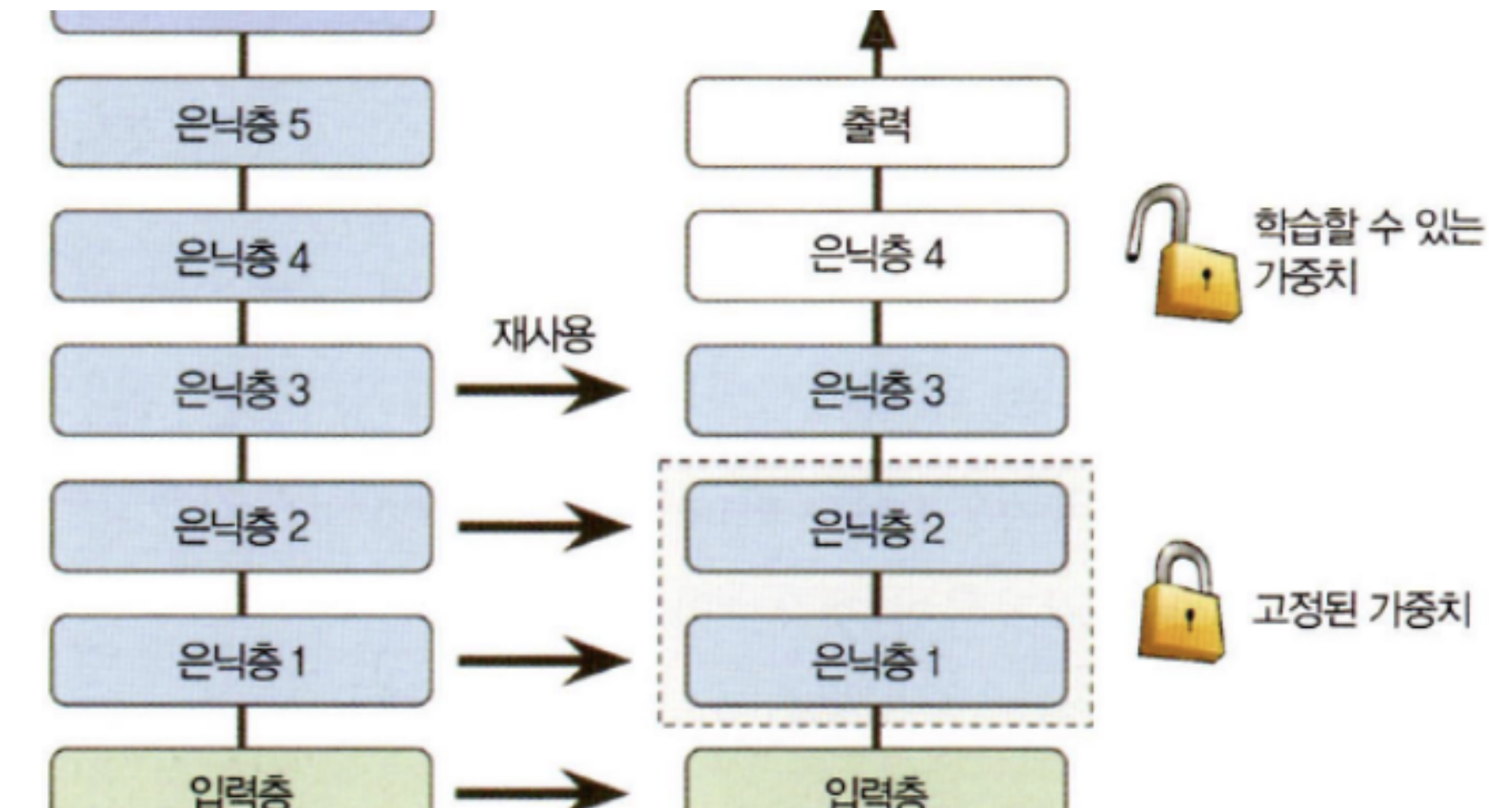
문제에 대한 분석

항목	항목의 세부 내용
문제의 입출력 식별, 데이터 구조 정의	입력: 전처리 된 MNIST 데이터셋(x_train, y_train) 출력: 학습 완료된 전이 학습 모델을 가지고 만든 테스트 데이터에 대한 정확도(test_acc). 데이터 구조: MNIST 데이터는 28×28 이미지를 (784,)로 펼쳐 정규화한 후 사용. 출력 레이블은 0~9의 정수값으로 구성됨.
입출력 변수 정의 및 관계식 도출	입력: x = MNIST 이미지 데이터 (784의 일차원 벡터), y = 레이블 데이터 (정수형, 0~9까지의 손글씨). 출력: test_acc(y_pred와 y_test와의 비교 후 정확도) 관계식 도출: 옵티마이저 = Adam(), 손실함수 = SparseCategoricalCrossentropy(), 평가지표 = accuracy(정확도)
문제 해결 제약 요구사항 고려	데이터 분할: train:test = 8:2 기존 모델 레이어 동결 : hidden1, hidden2 레이어는 동결하여 가중치를 재학습하지 않음.

알고리즘 분석



그래디언트 폭주문제 → 그래디언트 클리핑



폭주 문제가 안생긴 전이 모델의 레이어 동결을 통해
간접적으로 그래디언트 폭주 문제를 줄임

함수 및 모듈별 알고리즘 분석

- 그래디언트 클리핑을 통해 역전파 과정에서 특정 임계값을 넘지 못하도록 잘라냄
- 전이 모델을 통해 간접적으로 그래디언트 폭주 문제를 줄임
- 레이어를 동결하여 train 영향을 안받게 하고, 이후 원하는 레이어를 추가해 모델 확장

프로그램 작성(코딩)

1 기존 모델 생성 및 저장(base_model)

- MNIST 데이터를 통해 새로운 모델을 작성
- create_base_model 함수를 작성
- 입력층은 데이터를 1D벡터로 변환, 활성화 함수는 ReLU함수, 128개의 뉴런
- 은닉층 은 각각 뉴런 64, 32, 활성화 함수는 ReLU함수
- 출력층은 10개의 뉴런(0~9의 손글씨), 활성화함수는 softmax 함수
- x_train과 x_test를 전처리: 형태를 1D벡터로 맞추고, 픽셀값을 0~1로 맞춤
- 모델을 옵티마이저 'Adam', 손실함수 'SparsecategoricalCrossentropy'
- MNIST에서 y_train 레이블이 정수값으로 제공되므로 SparseCategoricalCrossentropy를 사용
- 모델 학습 후 모델 저장(my_model.h5)

```
# 기존 모델 생성 및 저장
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 기존 모델 생성
def create_base_model():
    model = Sequential([
        ## ReLU 함수를 사용해 활성화, input 형태는 784
        Dense(128, activation='relu', input_shape=(784,)), name='hidden1'),
        Dense(64, activation='relu', name='hidden2'),
        Dense(32, activation='relu', name='hidden3'),
        Dense(10, activation='softmax', name='output'),
    ])
    return model

# MNIST 데이터 로드 및 전처리
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test = x_test.reshape(-1, 784).astype('float32') / 255.0

# 모델 생성 및 컴파일
base_model = create_base_model()
base_model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)

# 모델 학습
base_model.fit(x_train, y_train, batch_size=64, epochs=5)

# 모델 저장
base_model.save('./my_model.h5') # 'my_model.ckpt'은 Keras에서 'h5' 형식으로 저장됩니다.
```

```
import tensorflow as tf
from tensorflow.keras.models import load_model, Model
from tensorflow.keras.layers import Dense, Input

base_model = load_model('./my_model.h5')
```

```
# 새로운 입력 레이어를 정의하세요. (입력 형태: (784,))
input_layer = Input(shape=(784,))

# 기존 모델에서 hidden1부터 hidden3까지를 가져와 연결하세요.
hidden1 = base_model.get_layer('hidden1')(input_layer)
hidden2 = base_model.get_layer('hidden2')(hidden1)
hidden3 = base_model.get_layer('hidden3')(hidden2)

# hidden1과 hidden2 레이어를 동결하세요.
for layer in [base_model.get_layer('hidden1'), base_model.get_layer('hidden2')]:
    layer.trainable = False

# 새로운 hidden4와 logits 레이어를 추가하여 모델을 확장하세요.
hidden4 = Dense(64, activation='relu', name='hidden4')(hidden3)
logits = Dense(10, activation='softmax', name='logits')(hidden4)

# 새로운 Transfer Learning 모델을 생성하세요.
transfer_model = Model(inputs=input_layer, outputs=logits)
```

2 사전 학습된 모델 load

- 만들어 두었던 사전학습 모델을 load하여 transfer_learning을 할 준비

3 사전 학습 모델을 활용한 새로운 모델 생성 Transfer_Learning(전이 학습)

- 새로운 모델의 입력층을 재정의 (1D벡터 형태의 형태)
- 기존 모델에서 새로운 모델의 입력층을 입력으로 받는 은닉층 생성 후 기존 모델의 은닉층 연결
- 기존 모델에서 학습된 가중치를 그대로 사용할 수 있도록 이 레이어들을 가져와 연결
- 은닉층 1(기존모델의 입력층), 은닉층 2를 동결
- 가중치를 학습하지 않게하여 이미 학습된 레이어 수정을 막아 학습시간을 줄임
- 새로운 은닉층 4, 출력층을 만들어 레이어 확장
- 새로운 모델을 만듦(Transfer_model, input을 새로운 모델의 입력층, output은 새로운 모델의 출력층)

4 새로운 모델 학습 및 평가

```
# 모델을 컴파일하세요. (옵티마이저: Adam, 손실 함수: Sparse Categorical Crossentropy, 메트릭: Accuracy)
transfer_model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)

# MNIST 데이터셋을 로드하고 전처리하세요. (데이터를 (784,)로 변환 및 정규화)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test = x_test.reshape(-1, 784).astype('float32') / 255.0

# 모델을 학습하세요. (배치 크기: 64, 에포크: 5, 검증 데이터: 20%)
history = transfer_model.fit(
    x_train, y_train,
    batch_size=64,
    epochs=5,
    validation_split=0.2
)

# 모델을 평가하고 테스트 정확도를 출력하세요.
test_loss, test_acc = transfer_model.evaluate(x_test, y_test, verbose=2)
print(f"테스트 정확도: {test_acc:.4f}")
```

- 옵티마이저를 'Adam', 손실함수를 SparseCategoricalCrossentropy, 평가지표는 accuracy를 추가
- MNIST 데이터셋을 로드하고, 각 train과 test를 구분 및 x에 대해 데이터 정규화
- 모델 학습 배치는 64크기로, 에포크는 5번 실행, 검증 데이터는 20%로 설정
- 모델의 손실값과 테스트 정확도를 evaluate 메소드를 통해 뽑음
- 그 후 테스트 정확도를 출력

실행결과

- base_model의 정확도: 0.9756
- transfer_model의 정확도: 0.9765

이로써 transfer_learning을 통해 입력 데이터를 줄이고

학습 시간을 줄일 수 있으며, 필요한 레이어는

동결시키며 가중치를 그대로 유지하며 그레이던트 폭주 문제를 방지할 수 있다.

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ----- 0s 0us/step
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
938/938 ----- 9s 7ms/step - accuracy: 0.8468 - loss: 0.5219
Epoch 2/5
938/938 ----- 8s 5ms/step - accuracy: 0.9636 - loss: 0.1204
Epoch 3/5
938/938 ----- 6s 6ms/step - accuracy: 0.9762 - loss: 0.0781
Epoch 4/5
938/938 ----- 3s 4ms/step - accuracy: 0.9809 - loss: 0.0597
Epoch 5/5
938/938 ----- 5s 4ms/step - accuracy: 0.9860 - loss: 0.0437
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model`
313/313 - 1s - 2ms/step - accuracy: 0.9756 - loss: 0.0800
테스트 정확도: 0.9756

```

```

Epoch 1/5
750/750 ----- 4s 4ms/step - accuracy: 0.8662 - loss: 0.5855 - val_accuracy: 0.9902 - val_loss: 0.0344
Epoch 2/5
750/750 ----- 4s 3ms/step - accuracy: 0.9903 - loss: 0.0322 - val_accuracy: 0.9909 - val_loss: 0.0293
Epoch 3/5
750/750 ----- 2s 3ms/step - accuracy: 0.9917 - loss: 0.0279 - val_accuracy: 0.9920 - val_loss: 0.0266
Epoch 4/5
750/750 ----- 3s 3ms/step - accuracy: 0.9922 - loss: 0.0263 - val_accuracy: 0.9914 - val_loss: 0.0279
Epoch 5/5
750/750 ----- 5s 3ms/step - accuracy: 0.9928 - loss: 0.0230 - val_accuracy: 0.9916 - val_loss: 0.0254
313/313 - 0s - 1ms/step - accuracy: 0.9765 - loss: 0.0842
테스트 정확도: 0.9765

```